23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No :A8
Team Members :

Roll No	Student Name
AM.SC.U4CSE25014	ADITHYAN B
AM.SC.U4CSE25024	ALI ADNAN THAHA
AM.SC.U4CSE25040	MADHAV.A
AM.SC.U4CSE25049	ROBIN ANTONY

Project Title : Smart Digital Library Management System

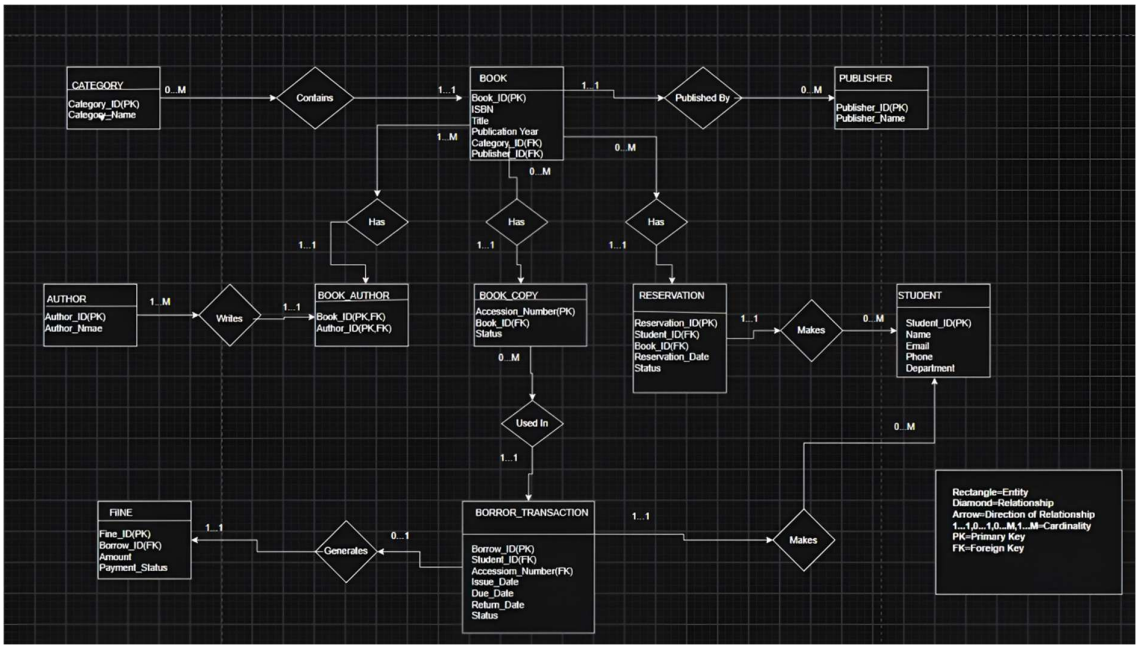
3.1. Detailed description of the project:

The **Smart Digital Library Management System (BookVerse)** is a database-driven system designed to manage the major activities of a college library. The system has two main users: **Library Administrator and Student**. The administrator manages library records, book circulation, reservations, overdue books, fines, and reports, while students can search for books, check availability, make reservations, and view their borrowing and fine history.

The database maintains information about **Students** (Student ID, name, email, phone, and department), **Books** (Book ID, title, ISBN, publication year, category, and publisher), **Book Copies** (accession number and status), **Authors**, **Publishers**, **Categories**, **Borrow Transactions**, **Reservations**, and **Fines**. A publisher and category can be associated with multiple books, a book can have multiple physical copies and authors, and the relationship between books and authors is maintained through **Book_Author**. A student can have multiple borrow transactions and reservations, while each borrow transaction is associated with one physical book copy and may generate a fine.

The system follows business rules such as maintaining unique IDs for records, ensuring that borrow transactions and reservations refer to existing students and books/copies, tracking the availability and status of physical copies, and recording fines for overdue borrow transactions. Primary keys and foreign keys are used to maintain data integrity and reduce redundancy. The system supports book search, issue and return operations, reservation management, automatic overdue and fine handling, CRUD operations, and report generation.

3.2. ER Diagram



3.3 Relational Schema

Relation	Attributes
STUDENT	**Student_ID (PK)**, Name, Email, Phone, Department
CATEGORY	**Category_ID (PK)**, Category_Name
PUBLISHER	**Publisher_ID (PK)**, Publisher_Name
BOOK	**Book_ID (PK)**, Title, ISBN, Publication_Year, Category_ID (FK), Publisher_ID (FK)
AUTHOR	**Author_ID (PK)**, Author_Name
BOOK_AUTHOR	**Book_ID (PK, FK)**, **Author_ID (PK, FK)**
BOOK_COPY	**Accession_Number (PK)**, Book_ID (FK), Status
Borrow_Transaction	**Borrow_ID (PK)**, Student_ID (FK), Accession_Number (FK), Issue_Date, Due_Date, Return_Date, Status

Relation	Attributes
RESERVATION	**Reservation_ID (PK)**, Student_ID (FK), Book_ID (FK), Reservation_Date, Status
FINE	**Fine_ID (PK)**, Borrow_ID (FK), Amount, Payment_Status

Here (Book_ID,Author_ID)forms the composite primary key.

3.4 Normalization and Functional dependency analysis:

3.4.1 Functional dependencies identified

The following functional dependencies are identified based on the proposed relational schema.

Relation	Functional Dependency
STUDENT	Student_ID → Name, Email, Phone, Department
CATEGORY	Category_ID → Category_Name
PUBLISHER	Publisher_ID → Publisher_Name
BOOK	Book_ID → Title, ISBN, Publication_Year, Category_ID, Publisher_ID
AUTHOR	Author_ID → Author_Name
BOOK_COPY	Accession_Number → Book_ID, Status
BORROW_TRANSACTION	Borrow_ID → Student_ID, Accession_Number, Issue_Date, Due_Date, Return_Date, Status
RESERVATION	Reservation_ID → Student_ID, Book_ID, Reservation_Date, Status
FINE	Fine_ID → Borrow_ID, Amount, Payment_Status
BOOK_AUTHOR	(Book_ID, Author_ID) is the composite key; there are no non-key attributes.

Significance of the FDs: each FD reflects a real-world business rule from the system description. For example, Accession_Number → Book_ID, Status encodes that a physical copy belongs to exactly one title and has one current status at a time; Borrow_ID → Student_ID, Accession_Number, … encodes that a single borrow record refers to exactly one student borrowing exactly one physical copy. The extra FD Borrow_ID → Fine_ID, … in FINE reflects the 0..1 cardinality between BORROW_TRANSACTION and FINE (at most one fine per borrow transaction), which makes Borrow_ID unique in FINE as well.

3.4.2 Candidate key justification (attribute closure)

STUDENT:

$\{Student_ID\}^+ = \{Student_ID, Name, Email, Phone, Department\}$

→ Student_ID is the candidate key.

CATEGORY:

$\{Category_ID\}^+ = \{Category_ID, Category_Name\}$

→ Category_ID is the candidate key.

PUBLISHER:

$\{Publisher_ID\}^+ = \{Publisher_ID, Publisher_Name\}$

→ Publisher_ID is the candidate key.

BOOK:

$\{Book_ID\}^+ = \{Book_ID, Title, ISBN, Publication_Year, Category_ID, Publisher_ID\}$

→ Book_ID is a candidate key.

$\{ISBN\}^+ = \{ISBN, Title, Publication_Year, Category_ID, Publisher_ID, Book_ID\}$

→ ISBN is also a candidate key.

→ BOOK has two candidate keys: {Book_ID} and {ISBN}. Book_ID is chosen as the primary key because it is a compact surrogate key, simpler to reference as a foreign key from BOOK_AUTHOR, BOOK_COPY, and RESERVATION.

AUTHOR:

$\{Author_ID\}^+ = \{ Author_ID, Author_Name\}$

→ Author_ID is the candidate key.

BOOK_COPY:

$\{Accession_Number\}^+ = \{Accession_Number, Book_ID, Status\}$

→ Accession_Number is the candidate key.

BORROW_TRANSACTION:

$\{Borrow_ID\}^+ = \{Borrow_ID, Student_ID, Accession_Number, Issue_Date, Due_Date, Return_Date, Status\}$

→ Borrow_ID is the candidate key.

RESERVATION:

$\{Reservation_ID\}^+ = \{Reservation_ID, Student_ID, Book_ID, Reservation_Date, Status\}$

→ Reservation_ID is the candidate key.

FINE:

$\{Fine_ID\}^+ = \{Fine_ID, Borrow_ID, Amount, Payment_Status\}$

→ Fine_ID is a candidate key.

$\{Borrow_ID\}^+ = \{Borrow_ID, Fine_ID, Amount, Payment_Status\}$ = all attributes (since each borrow transaction generates at most one fine)

→ Borrow_ID is also a candidate key.

→ FINE has two candidate keys: {Fine_ID} and {Borrow_ID}. Fine_ID is retained as the primary key for a stable surrogate identifier.

CATEGORY, PUBLISHER, and AUTHOR follow the same single-attribute-key pattern as STUDENT (Category_ID, Publisher_ID, Author_ID respectively, each with closure equal to all attributes).

3.4.3 Normalization (1NF → BCNF)

1NF

The one place this could have failed is the book–author relationship: a book can have multiple authors. Storing them as a comma-separated list inside BOOK would violate 1NF. Instead, the design resolves this many-to-many relationship into the separate BOOK_AUTHOR junction relation, so every relation — including BOOK_AUTHOR — has only atomic attributes. All relations satisfy 1NF.

2NF

The only relation with a composite key here is BOOK_AUTHOR (Book_ID, Author_ID), and it has no non-key attributes at all — so there is nothing that could be partially dependent on just one part of the key. Every other relation has a single-attribute candidate key, so partial dependency is impossible by definition. All relations satisfy 2NF.

3NF

In BOOK, Category_ID and Publisher_ID are foreign keys, not the category/publisher names themselves — Category_Name lives only in CATEGORY and Publisher_Name only in PUBLISHER. If BOOK instead stored Category_Name directly, we'd get a transitive dependency Book_ID → Category_ID → Category_Name . By keeping CATEGORY and PUBLISHER as separate relations and referencing them by ID, this transitive path is eliminated. The same reasoning applies to AUTHOR (Author_Name kept out of BOOK/BOOK_AUTHOR) and to BOOK_COPY/BORROW_TRANSACTION (Student and Book attributes are not duplicated there — only the FK is stored). All relations satisfy 3NF.

BCNF

Checking the determinant of every FD found in 3.4.1 against the candidate keys from 3.4.2:

Relation	**Determinant(s) in its FDs**	**Is it a candidate key?**

STUDENT	Student_ID	Yes
CATEGORY	Category_ID	Yes
PUBLISHER	Publisher_ID	Yes
BOOK	Book_ID, ISBN	Yes (both)
AUTHOR	Author_ID	Yes
BOOK_AUTHOR	(Book_ID, Author_ID)	Yes (only, trivial FD)
BOOK_COPY	Accession_Number	Yes
BORROW_TRANSACTION	Borrow_ID	Yes
RESERVATION	Reservation_ID	Yes
FINE	Fine_ID, Borrow_ID	Yes (both)

3.4.4 Conclusion

The proposed relational schema is normalized up to Boyce-Codd Normal Form (BCNF). Functional dependency analysis shows that each relation has a candidate key that determines its non-key attributes. Since all attributes are atomic, the relations satisfy 1NF. Partial dependencies are absent, satisfying 2NF. Transitive dependencies are eliminated by separating entities such as Category, Publisher, and Author into their respective relations, satisfying 3NF. Finally, every determinant of a non-trivial functional dependency is a superkey, so the relations satisfy BCNF.